Week 10 Level 2: Intermediate

Project Studio 1

Plan first, then build the core: a small program that works beats a big one that does not.

What you will learn

- How to turn your project plan into pseudocode and a flowchart.
- How to split your project into functions, and split the work fairly with your partner.
- How to build the **core loop** of your project: the smallest version that runs (Milestone M2).
- How to review a partner's code kindly and usefully.

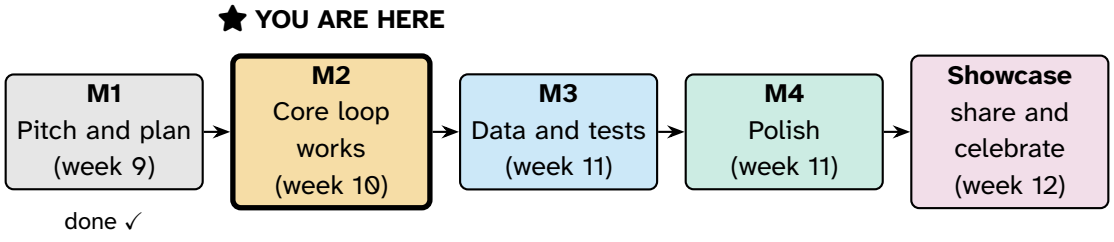
The big idea: build a skateboard first

If you want to build a car, you do not start with one perfect wheel. Nobody can ride a wheel! You build a skateboard first: tiny, but you can ride it. Then a scooter, a bike, a car. Every version works.

Your project works the same way. The **core loop** is your skateboard:

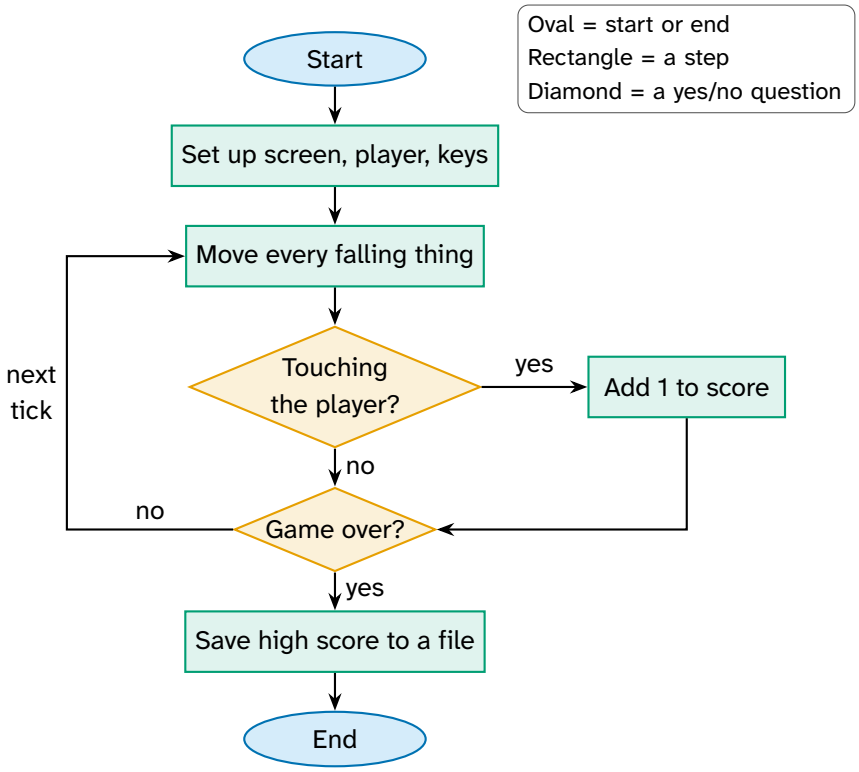
- **Arcade Game:** a window, a player that moves with the keys, and a timer loop that keeps running (`screen.ontimer`).
- **Art Studio:** a `while` loop that asks what to do (draw, save, load, quit) and does it.

No score, no title screen, no fancy colors yet. Those come later, and they are easy once the skeleton walks.



Pseudocode and flowcharts

Pseudocode is your program written in plain English, one step per line, before you write Python. A **flowchart** draws the same steps as shapes joined by arrows. Three shapes are all you need: **ovals** for start and end, **rectangles** for steps, and **diamonds** for yes/no questions. Here is the flowchart for a catching game:



The same plan as pseudocode:

Pseudocode

```
set up the screen, the player and the keys
repeat every tick:
    move every falling thing down
    if a falling thing touches the player: add 1 to the score
    if the game is over: save the high score and stop
```

Every line that does one job can become a **function**. “Move the player but not off the screen” becomes `next_x(x, direction)`: the position and direction go in, the new position comes out. That is **decomposition**: breaking a big problem into small named parts.

Exercise 10.1: Paper prototype and plan check (M1)

1. Draw your screen on paper. Use coins for things that move.
2. One partner is “the computer”, the other is the player. Play for one minute: every clap is one tick.
3. Finish your plan: write the pseudocode (or draw the flowchart) for your core loop below.

☐ My plan has a loop (every tick, or until Quit).

☐ My plan has at least one yes/no decision.

☐ My plan names the data: a list, a dictionary and a file.

Exercise 10.2: A fair task board

Write each function on its own sticky note. Fill in the table, then take turns choosing tasks: one each, like picking teams. Move notes from TO DO to DOING to DONE as you work.

Function name	What goes in	What comes out	Who	Done
`next_x`	x, direction	the new x		☐
				☐
				☐
				☐
				☐

Exercise 10.3: Build the core loop (M2)

Make a folder for your team’s project and save your program as `mygame.py` (or `mystudio.py`). Never call it `turtle.py`! Build it in small steps and press F5 after each one. For an arcade game,

the heart of the program is a loop that runs itself again:

the timer loop

```
def game_loop():
    if not state["running"]:
        return
    # TODO: move falling things, check for catches, add to the score
    screen.update()
    screen.ontimer(game_loop, 50)   # run again in 50 milliseconds
```

- ☐ My program runs without a red error.
- ☐ Something repeats by itself (it moves on the timer, or the menu comes back).
- ☐ At least one key or menu choice works.
- ☐ The start-up code is under `if __name__ == "__main__":`.
- ☐ Line 1 has a comment with our nicknames, and credit for any code we remixed.

Exercise 10.4: Partner code review

Swap seats with your partner (or another team). Run their program and read their code. Then write **two stars and a wish**: two things that are really good, and one kind idea. Be specific: "the name `next_x` tells me exactly what it does" helps more than "nice".

★

★

Wish:

Vocabulary box

pseudocode a plan for a program written in plain words, one step per line.

flowchart a drawing of a program: ovals for start and end, rectangles for steps, diamonds for yes/no questions.

decomposition breaking a big problem into small parts you can name and build one at a time.

core loop the part of a game or app that repeats over and over; the smallest version that works.

milestone a checkpoint on the way to the finished project, like M2 "core loop works".

code review reading someone else's code to help them find bugs and make it clearer.

Reflection

What was the smallest version of your project that worked today?

Was your task split fair? What would you change next week?

Week 10
Project Studio 1
Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

Tip: No computer at home this week?

That is fine. Do the paper parts (the dev log, the Challenge plan and the Creative sketch) and bring them next week. Your project files stay in your team folder; never email or upload them.

Core (everyone): Finish your own sticky notes (about 10 minutes)

1. Open your team's project in Thonny. Pick **one** sticky note from your TO DO column that is yours.
2. Build it in small steps. Press F5 after each step. Save with Cmd+S often.
3. If you get a red error, read the last line of the traceback and the line number it points to. Stuck for 5 minutes? Write down what you tried and stop: that is useful too.
4. Write a short **dev log** (developer diary) below.

What I worked on:

What broke, and how I fixed it (or what I tried):

What I will do next:

Challenge (want more?): Tests on paper, before the code

Next week you will write pytest tests. Get ahead: choose one of your functions that `returns` something and write three examples of what goes in and what should come out. Make at least one an **edge case** (a value at the very limit, like the edge of the screen or a score of 0).

Function and input	Expected answer	Normal or edge?
`next_x(0, 1)`	`20`	normal

Creative (make it yours): Title screen sketch

Design the title screen for your project on squared paper: its name, a picture, and one line that explains the keys ("← → to move, space to start"). Use colors from the Okabe–Ito palette (orange, sky blue, bluish green, yellow, blue, vermillion, reddish purple) and make sure nothing depends on color alone. Use your nickname, not your real name.

Self-check: tick each box when it is true

- ☐ I worked on one of my own tasks and saved my work.
- ☐ My program still runs (or I wrote down exactly where it breaks).
- ☐ I wrote my dev log: what I did, what broke, what is next.
- ☐ Anything I copied or remixed has a credit comment.